Alexandria University

Faculty of Engineering

Electrical Engineering Department

Elective1 (Measurements-2)

EEP 311

"Development of a Mobile Robot for Autonomous Ball Tracking and Multi-Mode Operation"

Presented by:

Team 2

Names

Gehad Mohamed Attia Mohamed Daabas

22010693

Shahd Ashraf Ahmed

22010845

Safia Ibrahim Eldawy

17010966

Abdalla Mohamed Abdel-Neeim

22010918

Kareema Ayman Hassan Barouma

22011845

Momen Essam Mahmoud

22011060

Mark Raouf Adly Ghaly

22011067

Mazen Ali Hossam

22011074

Mohamed Ayman Ahmed Mohamed

2101198

Mariam Amr Abdel-Ghani Moustafa Saafan

22011224

1. Abstract

A mobile robot system was designed and implemented using an ESP32-based embedded control unit, integrating mechanical design, embedded software, computer vision, and mobile application control. The robot was initially developed as a two-wheel self-balancing platform, where balancing was achieved using IMU-based orientation estimation with Digital Motion Processor (DMP) filtering, encoder feedback, and PID control. The embedded software was structured in a modular manner, including configuration definitions, encoder interrupt handling, IMU processing, motor driver control, yaw PID regulation, and Bluetooth communication. During experimental testing, limitations in motor torque affected the stability of the self-balancing system, particularly during motion and inclination. Due to cost and time constraints, the mechanical design was modified by adding a caster wheel to ensure stable movement while preserving system functionality. A computer vision system was implemented to detect and track a ball using a camera, enabling autonomous behavior. Additionally, a mobile application was developed to control the robot through three operating modes: manual control, chatbot-based commands, and ball-following mode. The final system demonstrates successful integration of embedded control, mechanical adaptation, vision-based perception, and user interaction within the constraints of a university-level project.

2. Introduction

Mobile robots have become an important platform for studying embedded control systems, autonomous navigation, and human–machine interaction. Advances in low-cost microcontrollers, sensors, and computer vision techniques have enabled the development of intelligent robotic systems capable of perceiving their environment and responding to user commands in real time.

The objective of this project is to design and implement a mobile robot that integrates embedded control, wireless communication, computer vision, and artificial intelligence–based user interaction. The robot is designed to operate in multiple modes, allowing manual control by the user, autonomous ball-following using a camera, and interaction through an AI-based chatbot interface.

The project focuses on practical implementation within the constraints of a university course, emphasizing modular embedded software design, mechanical stability, and system integration rather than industrial-level optimization. The proposed system demonstrates how different technologies can be combined into a single robotic platform to achieve flexible and intelligent behavior.

3. Embedded Control Software

3.1 Embedded System Overview

The ESP32 microcontroller is utilized as the central control unit of the robot. Sensor readings, motor actuation, and wireless communication are processed in real-time, enabling simultaneous execution of multiple tasks and maintaining smooth operation in all system modes.

3.2 Software Architecture and Libraries

The software is organized in a modular manner, with separate libraries developed for sensor processing, motor control, PID regulation, and communication. This structure allows independent testing and debugging of individual components, improving maintainability and code clarity.

3.3 Configuration Module

All system input/output pins, global variables, and constants are defined within the configuration module. This module centralizes hardware settings, which simplifies future modifications and reduces code duplication.

3.4 Encoder Processing Module

Motor encoders are handled using interrupt-driven routines. Counters are initialized at system startup, and real-time speed calculations are performed to provide accurate feedback for motor control and motion regulation.

3.5 IMU Processing Module

Orientation measurements are obtained from the Inertial Measurement Unit (IMU) sensor. A Digital Motion Processor (DMP) filter is applied to reduce noise and drift, producing more accurate angle estimations compared to conventional Kalman filtering methods.

3.6 Motor Driver Control Module

PWM signals are generated to control motor speed and direction. Frequency and duty cycles are adjusted to optimize low-speed performance, which ensures smooth acceleration and precise control during movement.

3.7 Motor Speed Correction Strategy

Differences in motor speeds, particularly at low velocity, are compensated through the application of a proportional correction factor. This ensures synchronized wheel motion and prevents deviation from the desired path.

3.8 Robot Motion Control Module

Motor and driver parameters are initialized, and a high-level control interface is provided for executing movement commands. Sensor feedback is integrated with motor actuation to achieve coordinated motion.

3.9 Motion Command Functions

Forward, backward, left, right, stop, start, and rotational movements are executed through predefined functions. These functions incorporate sensor data to maintain precise orientation and controlled navigation.

3.10 Bluetooth Communication Module

Wireless communication with the mobile application is maintained via Bluetooth. Mode selection and command transmission are handled reliably, ensuring low-latency and responsive control of the robot.

3.11 Testing Procedure

3.11.1IMU

Motors were tested individually with multiple motion types (forward, reverse, right, left) to ensure that they move properly.

3.11.2 Ticks per Turn

MPU6050 was tested separately to ensure the angles were read correctly

3.11.3 Correction Factor

By using the encoder, the motors were compared to determine which one is faster and the factor needed to compensate the unsimilarity.

3.11.4 Motors

To use the encoders to move a certain distance we needed to know each meter corresponds to how many ticks. the robot was moved for 3 seconds while the encoder was counting and the distance moved was measured by meter. by dividing number of ticks by distance moved, we get the constant ticks per meter.

4. Mechanical Design

4.1 Initial Self-Balancing Robot Concept

The robot was initially designed as a two-wheel self-balancing system, based on the inverted pendulum principle. The mechanical design was created using Fusion 360 software. The chassis consisted of two wooden

parts linked by screws: a front surface where components were mounted and a base supporting three battery holders. Each wheel was connected to a motor, which was attached to the battery holder. Stability was intended to be maintained by continuously adjusting wheel motion according to the tilt of the chassis, as measured by the IMU sensor.

4.2 Mechanical Structure and Layout

The chassis layout was designed to support all components while maintaining a low center of mass. Wheels and motors were positioned to ensure balanced weight distribution, and the assembly was optimized to reduce the risk of tipping.

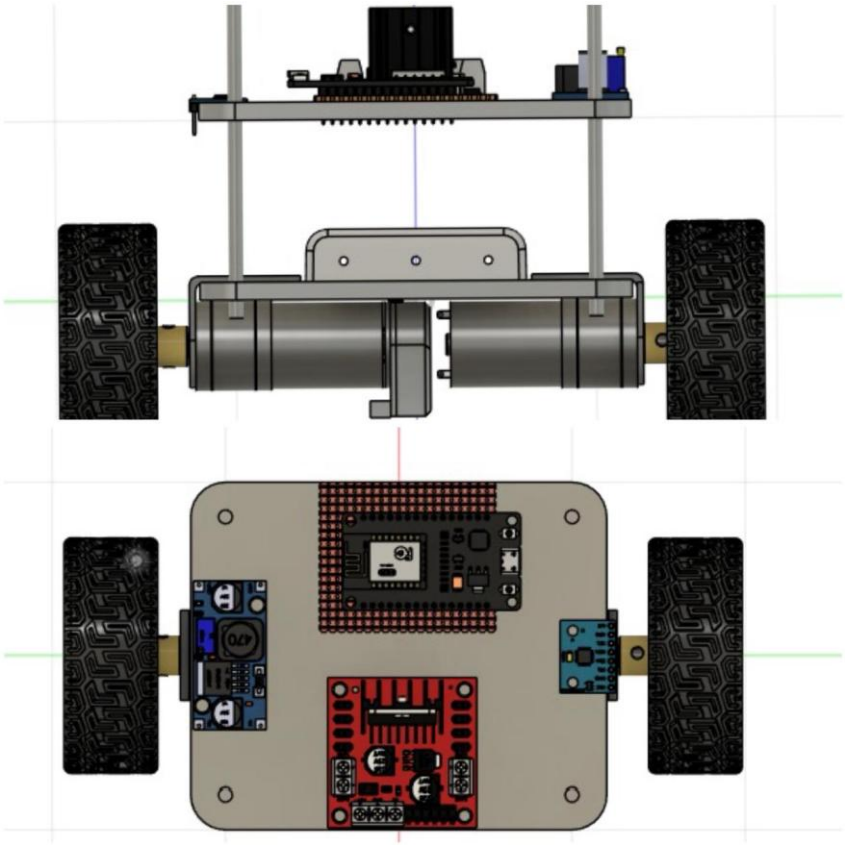

Figure 4.1 Side and Front View of Self-Balancing Robot

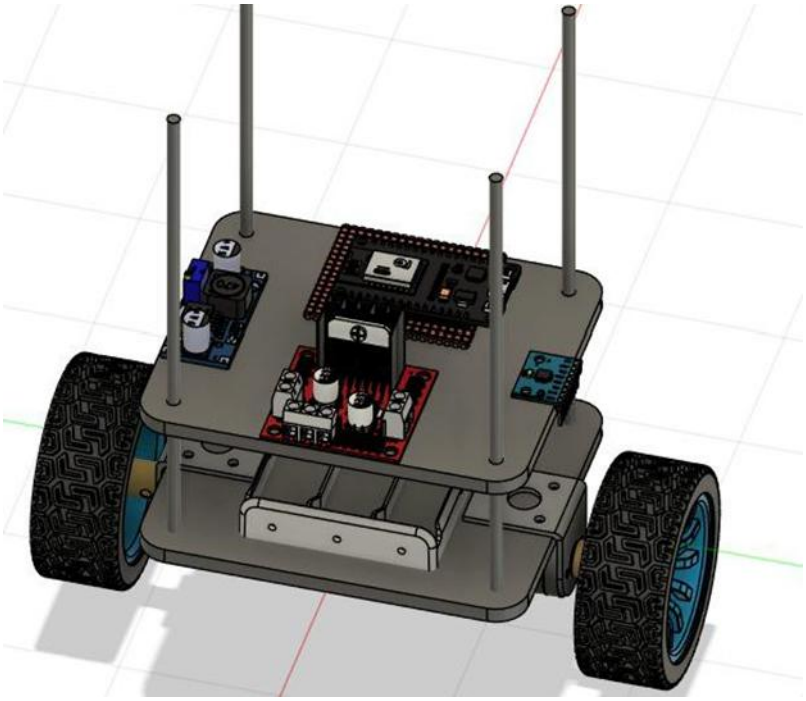

Figure 4.2 Overview on Self-Balancing Robot Design

4.3 Self-Balancing Control Limitation

During experimental testing, the robot exhibited instability when inclined or during motion. The system was able to maintain balance only for brief periods when stationary, indicating limitations in the original design.

4.4 Self-Balancing Control Implementation

The balancing of the robot was achieved using a PID (Proportional–Integral–Derivative) control strategy. Real-time tilt angles obtained from the IMU sensor were processed to generate corrective signals to the motors, maintaining vertical stability.

The proportional term (Kp) provided a response proportional to the tilt angle, allowing immediate correction of deviations from the vertical position. The integral term (Ki) accounted for the accumulation of tilt errors over time, eliminating steady-state offsets and ensuring long-term stability. The derivative term (Kd) responded to the rate of change of the tilt angle, damping oscillations and reducing overshoot.

PID parameters were tuned experimentally to balance responsiveness and stability. Excessively high Kp values caused oscillations, whereas

appropriate Ki and Kd values minimized steady-state errors and overshoot, resulting in smoother and more reliable balancing performance.

4.5 Identified Problem: Motor Torque Limitation

It was determined that the selected motors did not provide sufficient torque to sustain stability during movement or when external disturbances were applied. This limitation prevented reliable balancing and restricted operational performance.

4.6 Engineering Constraints

Budget and time restrictions prevented the selection of higher-torque motors. These constraints necessitated an alternative design approach to achieve mobility without compromising system stability.

4.7 Modified Mechanical Design Using Caster Wheel

The mechanical design was modified to overcome torque limitations while preserving mobility. The base surface was widened to increase stability, and a third caster wheel was added at the rear. The remainder of the structure, including the placement of motors, battery holders, and component mounting surfaces, remained unchanged from the original design. The addition of the caster wheel allowed free movement and provided passive stabilization, ensuring the robot could operate reliably.

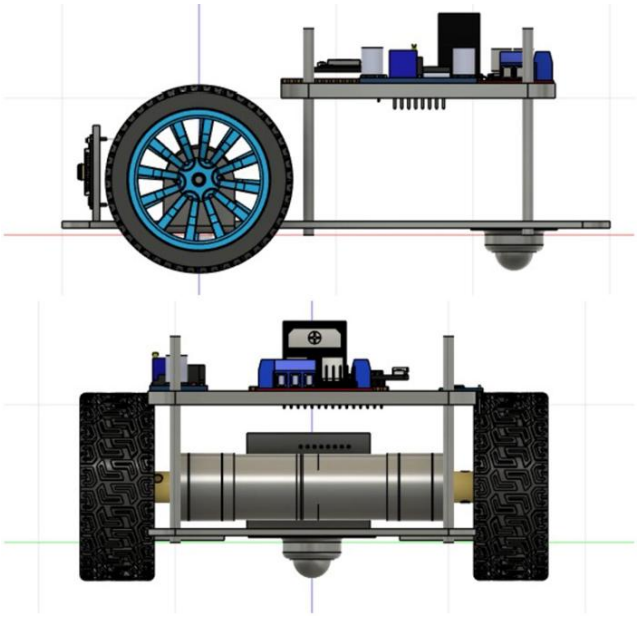

Figure 4.3 Other Point of View for Modified Designed

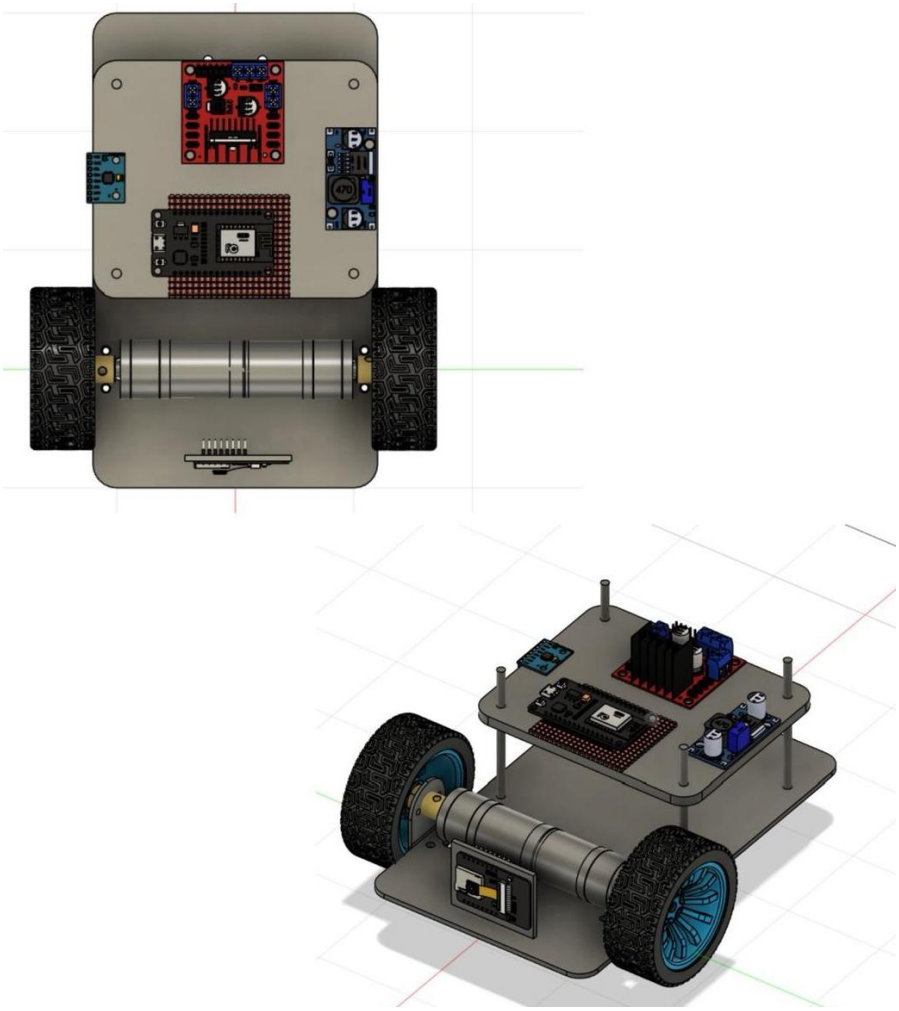

Figure 4.4 Modified Designed

4.8 Comparison Between Self-Balancing and Modified Design

The original self-balancing design exhibited limited stability and movement capabilities, particularly under external disturbances. In contrast, the modified design with the widened base and caster wheel allowed unrestricted movement and stable operation under all tested conditions. The design change was justified as an effective compromise between performance, cost, and project constraints.

5. Hardware Setup

5.1 Hardware System Overview

The hardware system was designed to support stable motion control, reliable sensing, and wireless communication while maintaining low cost and simplicity. The setup integrates a microcontroller-based control unit, motor actuation components, inertial sensing, power management circuitry, and auxiliary modules required for mobile robot operation. Emphasis was placed on component compatibility, availability, and suitability for real-time embedded and robotic applications.

5.2 Main Components Selection

The ESP32 microcontroller was selected as the main control unit due to its high processing capability, built-in Bluetooth connectivity, and low power consumption. These features make it suitable for embedded control, wireless communication with the mobile application, and coordination between sensing and actuation modules.

An IMU sensor was employed to provide orientation and angular motion data required for balance estimation and motion control. The selected IMU supports Digital Motion Processor (DMP) functionality, which enables accurate and stable attitude estimation with reduced computational load on the microcontroller.

Two GA25 DC gear motors with encoders were chosen to provide precise speed and position feedback. The integrated encoders enable closed-loop motor control and improve motion accuracy, which is essential for controlled navigation and stability.

The L298N motor driver module was used to interface the motors with the microcontroller. This driver allows bidirectional motor control and can handle the required voltage and current levels of the selected motors, making it appropriate for differential-drive mobile robots.

A buck converter (LM2596HVS) was included to step down the battery voltage to a stable 5 V supply. This ensures safe and reliable power delivery to low-voltage components such as the ESP32, IMU, and encoders.

Three 18650 lithium-ion batteries connected in series were selected as the main power source. These batteries provide sufficient energy density, rechargeability, and voltage levels suitable for mobile robotic applications.

5.3 Electrical Connections

The ESP32 was connected to the motor driver through digital control pins for direction control and PWM pins for speed regulation. Encoder outputs from each motor were connected to interrupt-capable pins on the ESP32 to enable accurate speed measurement. The IMU was interfaced with the microcontroller using an I²C communication protocol. The buck converter output was distributed to all low-voltage components, while the motor driver received power directly from the battery pack. A common ground was established among all components to ensure reliable signal reference and stable operation.

5.4 Power Calculations

Two voltage levels were used in the system. Three 18650 batteries connected in series provided approximately 12 V, which was supplied directly to the L298N motor driver and to the input of the buck converter. The buck converter output of 5 V was used to power the ESP32, IMU, encoders, and the logic circuitry of the motor driver. All grounds were connected together to maintain electrical consistency.

To estimate operating time, typical current consumption values were assumed based on standard datasheets. The ESP32 was assumed to draw approximately 200 mA during active operation with Bluetooth enabled. The IMU and encoders were estimated to draw approximately 50 mA combined. Each DC motor was assumed to draw an average current of 500 mA during normal operation, resulting in 1 A for both motors. The total average current consumption of the system was therefore estimated to be approximately 1.25 A.

With a battery capacity of 1500 mAh, the estimated operating time was calculated as:

$$Operating\ Time = \frac{1.5\ Ah}{1.25\ A} \approx 1.2\ hours$$

This estimation represents ideal conditions and does not account for current spikes, efficiency losses, or battery aging. In practical operation, the expected runtime is slightly lower but sufficient for experimental testing and demonstration purposes.

6. Computer Vision System

6.1 Vision System Overview

A computer vision system was implemented to detect a colored ball and provide real-time positional feedback for robot navigation. An external camera (Droid Cam) was mounted at the front of the robot to capture a forward-facing view of the environment. The live video feed was processed in real time using the OpenCV library, enabling continuous tracking of the target ball. Communication between the vision system and the ESP8266 microcontroller was established through a serial interface, allowing the transmission of motion commands and integration of perception with actuation.

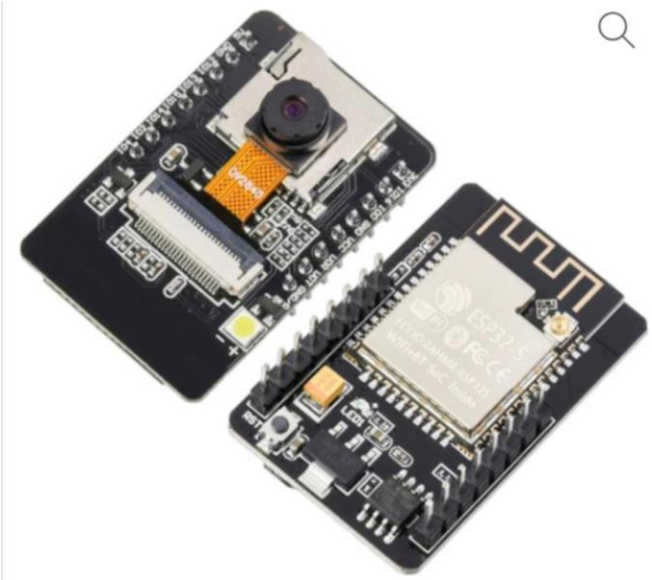

Figure 6.1 ESP32 CAM ESP32-S WIFI+BLUETOOTH With OV2640 Camera Module

6.2 Camera Setup and Placement

The camera was mounted at a fixed position at the front of the robot to ensure an unobstructed view of the path ahead. The mounting angle was selected to provide a clear perspective of the ground and the target object. This configuration allowed the ball to remain within the camera's field of view during robot movement, ensuring consistent detection and reliable guidance of navigation commands.

6.3 Image Processing Pipeline

Each captured video frame was processed through a sequence of steps to enhance detection accuracy. The frames were horizontally flipped to maintain intuitive directional control. They were then converted from RGB (BGR) to HSV color space, which offered improved robustness against lighting variations and facilitated precise color segmentation. A binary mask corresponding to the target ball color was generated to isolate the ball from the background. Gaussian blurring was applied to reduce noise and stabilize subsequent shape detection. Circular objects corresponding to the ball were identified using the Hough Circle Transform, and the center coordinates and radius of each detected object were extracted for further analysis.

6.4 Ball Detection Algorithm

The ball detection algorithm relied on color-based segmentation combined with circular shape recognition. The horizontal position of the detected ball was mapped to three predefined regions within the camera frame: left, center, and right. Based on the ball's location, movement commands were generated, with leftward, forward, or rightward motion corresponding to the respective regions. Additionally, the apparent size of the ball in pixels was converted into approximate real-world distance using a calibration factor, allowing the robot to estimate its distance from the ball. The positional information was transmitted to the robot through the serial interface, enabling automated control of its movement.

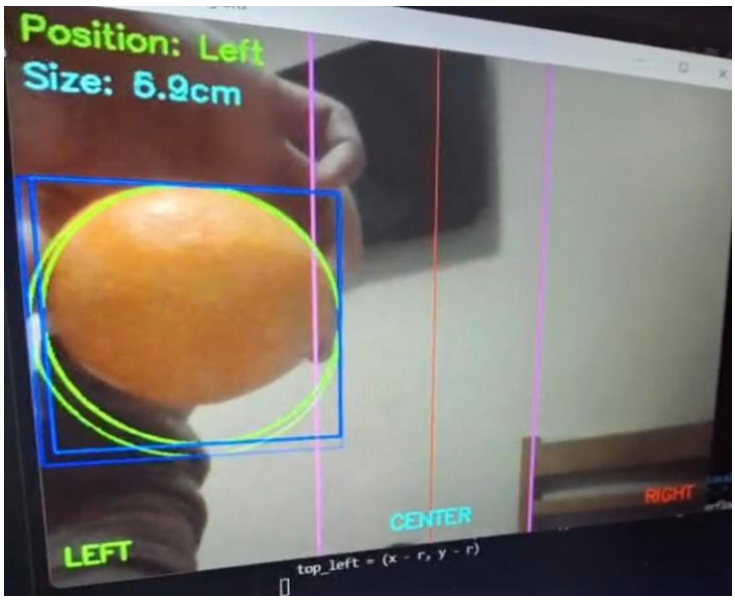

Figure 6.2 Camera User Interface

6.5 Performance and Limitations

The vision system demonstrated reliable detection and tracking of the target ball in real time under controlled lighting conditions. The division of the camera frame into three regions simplified navigation decisions and reduced computational complexity. The system was robust under moderate variations in lighting and was efficiently integrated with the robot's motor control via serial communication. However, detection accuracy was affected by significant lighting changes and background colors similar to the target ball, while distance estimation remained approximate and dependent on camera calibration. Performance was observed to degrade in cluttered or outdoor environments. Overall, the computer vision system provided effective autonomous ball-following behavior and functioned as a reliable perception module for the robot navigation framework.

7. Mobile Application and User Interface

7.1 Mobile Application Overview

A mobile application was developed to provide user interaction and control of the robot. The application supports multiple modes of operation, allowing the user to select between manual control, chatbot-based commands, and autonomous ball-following. The interface was designed to be intuitive and responsive, enabling efficient communication with the robot in real time.

7.2 Communication with the Robot

Communication between the mobile application and the robot was established using a Bluetooth interface. The application transmitted commands to the ESP32 microcontroller, which interpreted the data and executed corresponding movement actions. The wireless connection allowed for flexible operation and real-time feedback of robot status.

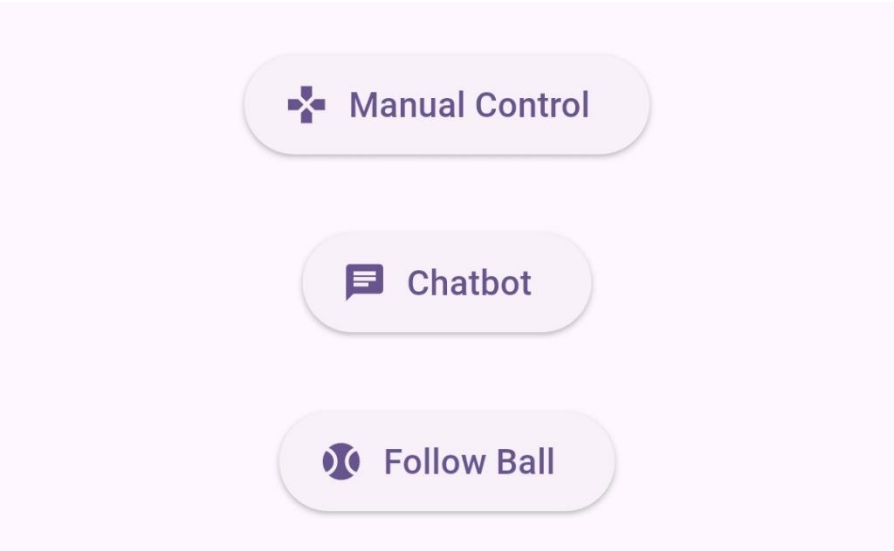

Figure 7.1 Robot Control Home

7.3 Chatbot Control Mode

The chatbot control mode allowed commands to be issued through text-based interaction with the mobile application. User input was processed by the application and translated into corresponding robot actions. This mode demonstrated the capability of integrating simple artificial intelligence-based interaction with embedded control, enhancing the versatility of the system.

7.4 Manual Control Mode

In the manual control mode, the user directly controlled the robot's movements through the application interface. Commands for forward, backward, left, right, start, stop, and rotation were transmitted via Bluetooth and executed by the robot. This mode provided a direct method for testing the responsiveness of the robot and verifying the integration of sensors and motors.

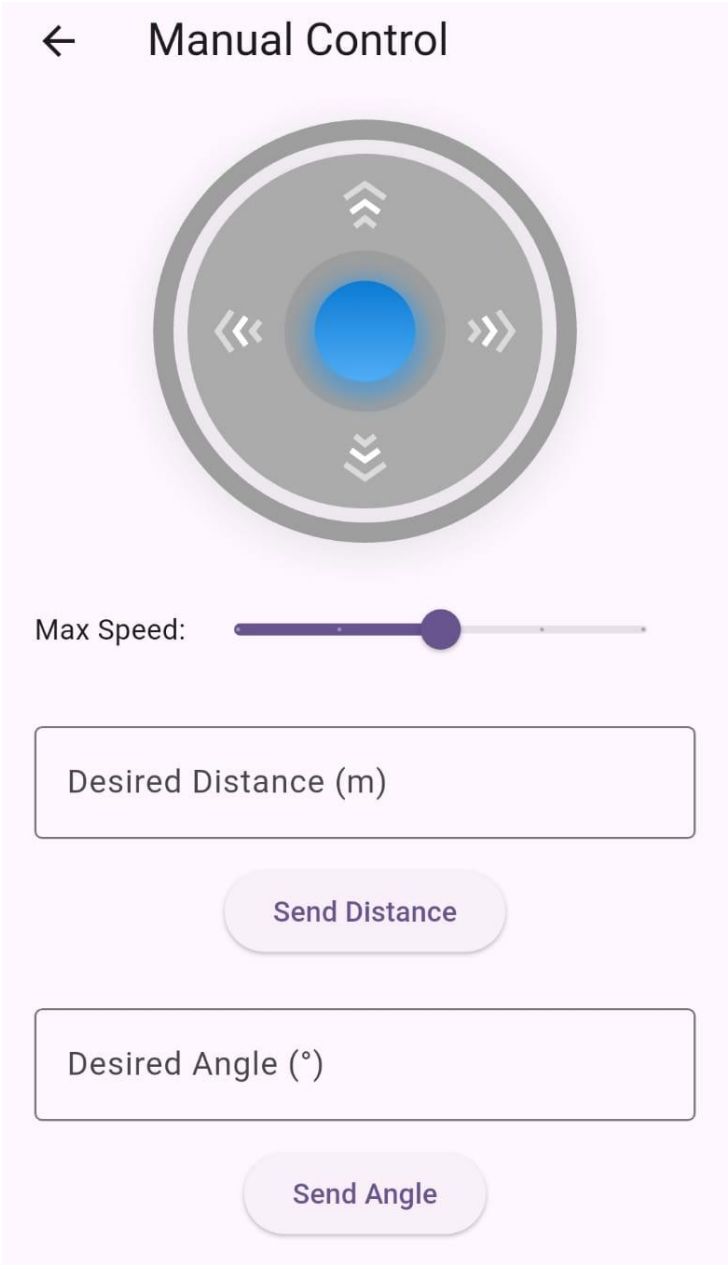

Figure 7.2 Manual Control Interface

7.5 Ball Following Mode

The ball-following mode was designed to activate the program previously uploaded to the microcontroller, enabling the robot to autonomously follow the target ball. Upon selection of this mode in the mobile application, the robot executed the computer vision functions described in Section 6. The vision system processed camera frames in real time, identified the position of the ball, and generated corresponding movement commands. These commands were transmitted to the robot's

motors via the embedded control system, allowing it to follow the ball according to the calculated position and distance. The mobile application provided monitoring and feedback of the robot's actions, while the microcontroller handled all real-time execution of navigation and control based on visual input.

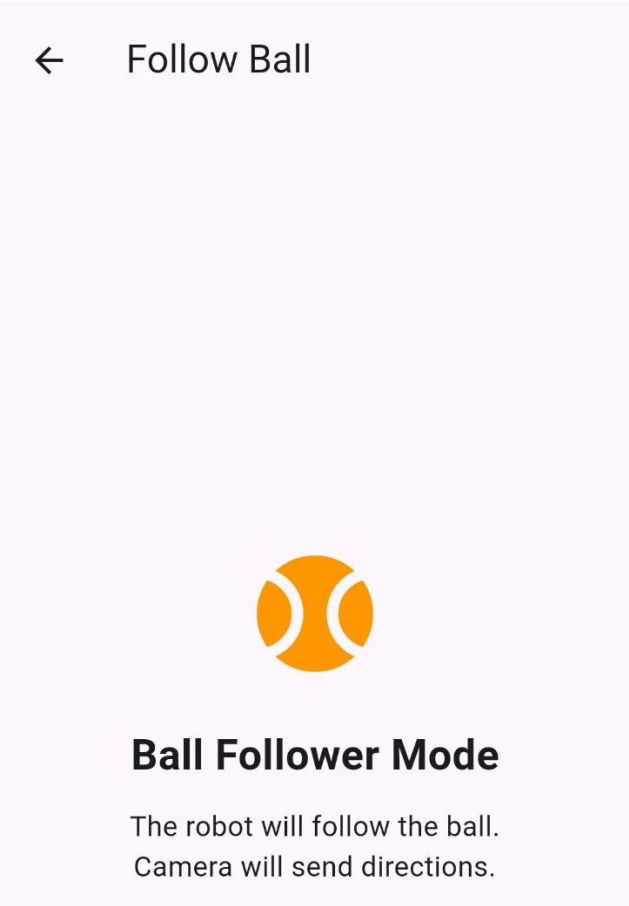

Figure 7.3 Follow Ball Interface

8. Cost Estimation

The overall cost of the project was calculated by summing the expenses of all individual components used in the construction of the robot. A detailed table was prepared to list each purchased item, including the microcontroller, motors, motor drivers, sensors, camera, mechanical parts, and additional hardware. The table provides the quantity, unit cost, and total cost for each component, allowing a clear overview of the project's budget. This cost analysis facilitates evaluation of resource allocation and provides a reference for potential future improvements or replication of the system.

Table1 Cost Estimation

Component Name	Price Each	Quantity	Total Price
ESP32	295 EG	1	**295 EG**
10 cm Screws	12.5 EG	4	**50 EG**
IMU	125 EG	1	**125 EG**
25GA370 DC Gear Motor With Encoder 250 RPM 12Vdc	475 EG	2	**950 EG**
Ropot Wheel 65mm *26mm +Metal Coupler 4mm	165 EG	2	**330 EG**
Reachargeable 18650 Li-ion Battary 3.7 V 1500mAh	50 EG	3	**150 EG**
LM2596HVS DC-DC Buck Converter 3A Step Down Module	70 EG	1	**70 EG**
18650 Battery Holder 3-slot	18 EG	1	**18 EG**
L298 Motor Driver Module	75 EG	1	**75 EG**
Mounting Bracket 25mm for GA25 DC	45 EG	2	**90 EG**
Caster Wheel	25 EG	1	**25 EG**
Total			**2178 EG**

9. Conclusion

A multifunctional mobile robot was successfully developed, integrating embedded control, mechanical design, computer vision, and mobile application interfaces. The ESP32 microcontroller managed modular software libraries for sensor processing, motor control, PID-based balancing, and Bluetooth communication. Self-balancing was

implemented using a PID controller with tuned proportional, integral, and derivative gains, while the mechanical design was modified to include a caster wheel to address torque limitations and improve stability.

The computer vision system detected and tracked a colored ball in real time using color segmentation and shape analysis, generating movement commands executed by the microcontroller. The mobile application enabled manual control, chatbot-based interaction, and activation of the autonomous ball-following mode. Real-time performance and robust operation were achieved under moderate lighting conditions, although limitations were observed in balancing under low motor torque, sensitivity to lighting, and approximate distance estimation.

Overall, the system demonstrated reliable integration of hardware, software, and vision-based navigation, providing a practical foundation for future enhancements in autonomous mobile robotics.

10. Recommendation

Future improvements could include the use of higher-torque motors to enable the implementation of the original two-wheel self-balancing design. Increasing motor torque would allow the PID controller to maintain stability during motion and under external disturbances, eliminating the need for a caster wheel. Additionally, enhancements in sensor calibration, computer vision algorithms, and PID tuning could further improve balancing performance, navigation accuracy, and robustness under varying lighting and environmental conditions. These modifications would provide a more autonomous and efficient mobile robot while preserving the compact design and functionality of the current system.

11. Appendix

11.1 GitHub

GitHub/balancing_robot